

main.py — Line-by-Line Code Explanation

AI Multi-Object Detection + Helmet Detection + Line-Crossing Counter

This document explains, section by section, exactly what every part of main.py does. The script runs two YOLO models at once on a live webcam feed: a general COCO object detector (with tracking + a person line-crossing counter) and a custom-trained helmet detector, then overlays all results on screen and saves the annotated video.

1. Imports

```
from ultralytics import YOLO
import cv2
import time
```

- `from ultralytics import YOLO` — imports the YOLO class from the Ultralytics library, used to load and run YOLOv8 models (both for detection and tracking).
- `import cv2` — imports OpenCV, used for reading the webcam feed, drawing boxes/text/lines on frames, and writing the output video file.
- `import time` — imports Python's time module, used here to measure how long each frame takes to process, in order to calculate FPS (frames per second).

2. Loading the Models

```
general_model = YOLO("yolov8m.pt")
helmet_model = YOLO("runs/detect/train/weights/best.pt")
```

- `general_model = YOLO("yolov8m.pt")` — loads the pretrained YOLOv8 'medium' model. This model already knows 80 everyday object classes from the COCO dataset (person, car, bottle, chair, etc.) — no custom training needed for this one.
- `helmet_model = YOLO("runs/detect/train/weights/best.pt")` — loads a custom-trained model, specifically the 'best' checkpoint saved from a previous training run. This is the model trained to detect helmets ('helmet' vs 'no-helmet', based on the class names used later in the code).

Note: *Two separate YOLO models run on every frame in this script — one general-purpose, one custom — which is why performance settings like `imgsz` and `half precision` are tuned carefully further down.*

3. Setting Up the Webcam

```
cap = cv2.VideoCapture(0)

frame_width = 640
frame_height = 480

cap.set(3, frame_width)
cap.set(4, frame_height)
```

- `cap = cv2.VideoCapture(0)` — opens the default webcam (device index 0). If you have multiple cameras, changing this number selects a different one.

- `frame_width = 640 / frame_height = 480` — defines the resolution the video will be captured and processed at.
- `cap.set(3, frame_width)` — property ID 3 in OpenCV corresponds to `CAP_PROP_FRAME_WIDTH`, so this sets the capture width to 640.
- `cap.set(4, frame_height)` — property ID 4 corresponds to `CAP_PROP_FRAME_HEIGHT`, so this sets the capture height to 480.

4. Setting Up the Output Video Writer

```
out = cv2.VideoWriter(
    "output.avi",
    cv2.VideoWriter_fourcc(*'XVID'),
    20,
    (frame_width, frame_height)
)
```

- `cv2.VideoWriter(...)` — creates a video writer object that will save the processed (annotated) frames to a file.
- `"output.avi"` — the output filename/path; every processed frame gets appended to this file.
- `cv2.VideoWriter_fourcc(*'XVID')` — specifies the video codec (XVID) used to encode the .avi file.
- `20` — the output frame rate (20 frames per second) that the saved video will play back at.
- `(frame_width, frame_height)` — the resolution of the output video, matching the webcam capture size (640x480).

5. Line-Crossing Counter Setup

```
line_x = 320

crossed_ids = set()
total_count = 0
```

- `line_x = 320` — the X-coordinate (horizontal pixel position) of a vertical counting line drawn across the middle of a 640-wide frame.
- `crossed_ids = set()` — an empty set used to remember which tracked person IDs have already crossed the line, so the same person isn't counted twice.
- `total_count = 0` — a running total of how many unique people have crossed the line so far.

6. FPS Tracking Variables

```
prev_time = 0
frame_count = 0
```

- `prev_time = 0` — stores the timestamp of the previously processed frame, used to calculate elapsed time between frames (and therefore FPS).
- `frame_count = 0` — counts how many frames have been read from the webcam so far; used below to skip every other frame for performance.

7. Main Loop — Reading Frames

```
while True:
    ret, frame = cap.read()
    if not ret:
        break
    frame = cv2.resize(frame, (640, 480))
    frame_count += 1
    if frame_count % 2 != 0:
        continue
```

- `while True:` — starts an infinite loop that keeps running until manually broken (by pressing 'q' or if the camera feed fails).
- `ret, frame = cap.read()` — reads one frame from the webcam. `ret` is `True` if the read succeeded, and `frame` is the actual image data.
- `if not ret: break` — if the camera failed to provide a frame (e.g., disconnected), the loop stops.
- `frame = cv2.resize(frame, (640, 480))` — forces every frame to exactly 640x480, regardless of the camera's native resolution, to keep processing consistent.
- `frame_count += 1` — increments the frame counter by 1 on every loop iteration.
- `if frame_count % 2 != 0: continue` — this is a performance optimization: it skips processing on every odd-numbered frame (i.e., only every 2nd frame is actually run through the AI models), which roughly doubles the effective speed at a small cost to smoothness.

8. FPS Calculation

```
current_time = time.time()
fps = 1 / (current_time - prev_time) if prev_time != 0 else 0
prev_time = current_time
```

- `current_time = time.time()` — captures the current timestamp (in seconds) at the start of processing this frame.
- `fps = 1 / (current_time - prev_time) if prev_time != 0 else 0` — calculates frames-per-second as the inverse of the time elapsed since the last processed frame. The '`if prev_time != 0`' guards against a divide-by-zero error on the very first frame.
- `prev_time = current_time` — updates `prev_time` so the next loop iteration can calculate the next FPS value correctly.

9. Running the General (COCO) Object Detector

```
general_results = general_model.track(
    frame,
    persist=True,
    imgsz=640,
    conf=0.4,
    half=True
)
```

- `general_model.track(...)` — runs YOLO's built-in object tracker (not just detection) on the frame. Unlike `.predict()`, `.track()` assigns a persistent ID to each detected object across frames, which is essential for the line-crossing counter to work.

- frame — the current webcam image being analyzed.
- persist=True — tells YOLO to remember tracked objects' IDs between calls, so the same person keeps the same ID as they move across frames (instead of getting a new ID every frame).
- imgsz=640 — resizes the input internally to 640x640 for the model's inference (a standard YOLO input size, balancing speed and accuracy).
- conf=0.4 — the confidence threshold: only detections the model is at least 40% sure about are kept. Comment in the code notes this was increased from 0.5 originally — actually it was lowered, which allows slightly more detections through, but the developer's comment indicates intent to reduce false positives.
- half=True — enables FP16 (half precision) inference, which roughly doubles inference speed on a compatible GPU by using 16-bit floats instead of 32-bit (has no effect / may error on CPU-only systems).

10. Running the Custom Helmet Detector

```
helmet_results = helmet_model.predict(
    frame,
    imgsz=320,
    conf=0.65,
    half=True
)
```

- helmet_model.predict(...) — runs a single-shot detection pass (no tracking/IDs needed here) using the custom-trained helmet model.
- imgsz=320 — uses a smaller internal input size (320x320) than the general model, trading some accuracy for extra speed, since running two models per frame is expensive.
- conf=0.65 — a stricter confidence threshold (65%) to filter out false positives, e.g., the code comment notes this helps avoid misclassifying things like hair as a helmet.
- half=True — same FP16 speed optimization as the general model.

11. Drawing the Counting Line

```
cv2.line(
    frame,
    (line_x, 0),
    (line_x, 480),
    (0, 0, 255),
    3
)
```

- cv2.line(...) — draws a straight line directly onto the frame.
- (line_x, 0) to (line_x, 480) — the line runs vertically from the top of the frame (y=0) to the bottom (y=480), at the fixed x-position 320 (the middle of the frame).
- (0, 0, 255) — the line's color in BGR format (OpenCV uses Blue-Green-Red order, not RGB) — this is pure red.
- 3 — the line thickness in pixels.

12. Processing General (COCO) Detections

```
object_counts = {}

for result in general_results:
    boxes = result.bboxes
    for box in boxes:
        if box.id is None:
            continue
        track_id = int(box.id[0])
        class_id = int(box.cls[0])
        class_name = general_model.names[class_id]
        confidence = float(box.conf[0])
        x1, y1, x2, y2 = map(int, box.xyxy[0])
        center_x = (x1 + x2) // 2
        center_y = (y1 + y2) // 2
```

- `object_counts = {}` — an empty dictionary created fresh each frame, used to tally how many of each object class (person, car, chair, etc.) are visible in the current frame.
- `for result in general_results:` — YOLO returns a list of results (one per image passed in); since only one frame is passed, this loop runs once.
- `boxes = result.bboxes` — extracts all the detected bounding boxes from this result.
- `for box in boxes:` — loops through every individual detected object in the frame.
- `if box.id is None: continue` — skips any detection that wasn't assigned a tracking ID (can happen briefly when a new object first appears before the tracker locks on).
- `track_id = int(box.id[0])` — extracts this object's unique tracking ID number.
- `class_id = int(box.cls[0])` — extracts the numeric class ID (e.g., 0) that YOLO predicted for this object.
- `class_name = general_model.names[class_id]` — converts the numeric class ID into a human-readable name (e.g., 'person', 'car') using the model's built-in class name list.
- `confidence = float(box.conf[0])` — extracts how confident the model is in this detection, as a decimal between 0 and 1.
- `x1, y1, x2, y2 = map(int, box.xyxy[0])` — extracts the bounding box coordinates: top-left corner (x1, y1) and bottom-right corner (x2, y2), converted to integers for pixel positions.
- `center_x = (x1 + x2) // 2` — calculates the horizontal center point of the bounding box (used to check line-crossing).
- `center_y = (y1 + y2) // 2` — calculates the vertical center point of the bounding box (calculated but not actually used elsewhere in this script).

13. Counting Objects & Drawing Boxes

```
if class_name not in object_counts:
    object_counts[class_name] = 0
object_counts[class_name] += 1

cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)

cv2.putText(
    frame,
```

```
f"{class_name} {confidence:.2f}",
(x1, y1 - 10),
cv2.FONT_HERSHEY_SIMPLEX,
0.5,
(0, 255, 0),
2
)
```

- if `class_name` not in `object_counts`: `object_counts[class_name] = 0` — initializes the count for a class the first time it's seen in this frame.
- `object_counts[class_name] += 1` — increments the tally for this object class.
- `cv2.rectangle(...)` — draws a green ((0, 255, 0) in BGR) bounding box rectangle around the detected object, 2 pixels thick.
- `cv2.putText(...)` — writes a text label above the box showing the class name and confidence score (e.g., 'person 0.87'), in green, using OpenCV's default simplex font at 0.5 scale and 2px thickness.

14. Person Line-Crossing Logic

```
if class_name == "person":
    if center_x > line_x:
        if track_id not in crossed_ids:
            crossed_ids.add(track_id)
            total_count += 1
            print(f"Person {track_id} crossed line")
```

- if `class_name == "person"`: — this crossing-counter logic only applies to objects classified as 'person' (ignores cars, chairs, etc.).
- if `center_x > line_x`: — checks whether the center of this person's bounding box is to the right of the counting line (`x > 320`).
- if `track_id not in crossed_ids`: — checks whether this specific tracked person has already been counted before, to avoid double-counting the same person every frame they remain past the line.
- `crossed_ids.add(track_id)` — marks this person's ID as having crossed, so they won't be counted again.
- `total_count += 1` — increments the overall crossing counter.
- `print(f"Person {track_id} crossed line")` — logs a message to the console/terminal each time a new person crosses, useful for debugging.

15. Processing Helmet Detections

```
for result in helmet_results:
    boxes = result.boxes
    for box in boxes:
        class_id = int(box.cls[0])
        confidence = float(box.conf[0])
        class_name = helmet_model.names[class_id]
        x1, y1, x2, y2 = map(int, box.xyxy[0])

        color = (255, 0, 0)
        if "no" in class_name.lower():
            color = (0, 0, 255)
```

- for result in helmet_results: / for box in boxes: — loops through every helmet-model detection in the frame, same pattern as the general model above.
- class_id, confidence, class_name, x1/y1/x2/y2 — extracted the same way as before, but using helmet_model.names for the label lookup (so classes here would be something like 'helmet' and 'no-helmet', based on how the custom dataset was labeled).
- color = (255, 0, 0) — default box color is blue (BGR), used for a positive 'helmet' detection.
- if "no" in class_name.lower(): color = (0, 0, 255) — if the detected class name contains 'no' (e.g., 'no-helmet' / 'no_helmet'), the box color is switched to red instead, to visually flag safety violations.

16. Drawing Helmet Detection Boxes

```
cv2.rectangle(frame, (x1, y1), (x2, y2), color, 3)

cv2.putText(
    frame,
    f"{class_name} {confidence:.2f}",
    (x1, y1 - 10),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    color,
    2
)
```

- cv2.rectangle(...) — draws the bounding box in blue (helmet) or red (no helmet), 3 pixels thick — slightly thicker than the general object boxes so helmet violations stand out.
- cv2.putText(...) — labels the box with the class name and confidence, in the matching color, at a slightly larger 0.6 font scale.

17. Displaying Live Object Counts On-Screen

```
y_position = 30
for obj_name, count in sorted(object_counts.items()):
    cv2.putText(
        frame,
        f"{obj_name}: {count}",
        (20, y_position),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.5,
        (255, 255, 255),
        1
    )
    y_position += 22
    if y_position > 400:
        break
```

- y_position = 30 — starting vertical pixel position for the first line of text (top-left corner of the frame).
- for obj_name, count in sorted(object_counts.items()): — loops through every object class detected this frame, alphabetically sorted, printing a running tally like 'car: 2', 'person: 3'.
- cv2.putText(...) — draws each count as white text at the current y_position.

- `y_position += 22` — moves down 22 pixels for the next line of text, so each class gets its own line.
- `if y_position > 400: break` — stops adding more text lines once they'd start overlapping the bottom status text (FPS/cross count), preventing the list from running off-screen.

18. Displaying Cross Count & FPS

```
cv2.putText(
    frame,
    f"Cross Count: {total_count}",
    (20, 430),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.7,
    (0, 0, 255),
    2
)

cv2.putText(
    frame,
    f"FPS: {int(fps)}",
    (20, 460),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.7,
    (255, 255, 0),
    2
)
```

- First `cv2.putText(...)` — displays the total number of people who have crossed the line so far, in red text, near the bottom-left of the frame (fixed position `y=430`).
- Second `cv2.putText(...)` — displays the current FPS (rounded to a whole number) in cyan/yellow (`(255, 255, 0)` in BGR), just below the cross count (`y=460`), so you can monitor real-time performance.

19. Saving & Showing the Frame

```
out.write(frame)

cv2.imshow(
    "Multi Object + Helmet Detection System",
    frame
)

if cv2.waitKey(1) & 0xFF == ord("q"):
    break
```

- `out.write(frame)` — writes the fully annotated frame (with boxes, lines, counts, FPS) to the `output.avi` video file.
- `cv2.imshow(...)` — opens/updates a live preview window titled 'Multi Object + Helmet Detection System', showing the annotated frame in real time.
- `cv2.waitKey(1)` — waits 1 millisecond for a keyboard key press (also required for OpenCV's window to refresh/render properly).
- `& 0xFF == ord("q")` — checks whether the pressed key was 'q'; if so, `break` exits the main while loop, ending the program.

20. Cleanup

```
cap.release()
out.release()
cv2.destroyAllWindows()
```

- `cap.release()` — releases the webcam so other programs can use it again.
- `out.release()` — finalizes and closes the `output.avi` video file, ensuring it's properly saved and playable.
- `cv2.destroyAllWindows()` — closes any OpenCV preview windows that were opened during the program's run.

Summary: What This Script Actually Does

- Opens the webcam and captures live video at 640x480.
- Runs two YOLOv8 models on every 2nd frame: a general COCO detector (with tracking) and a custom helmet detector.
- Draws a vertical counting line in the middle of the frame and counts how many unique people cross it (left to right), using tracked IDs to avoid double-counting.
- Flags helmet vs. no-helmet detections in different colors (blue vs. red) for safety monitoring.
- Displays live counts of every detected object class, the total person-crossing count, and the current FPS, all overlaid on the video.
- Saves the entire annotated video to `output.avi` and shows a live preview window until 'q' is pressed.

Quick Reference — Key Variables

Variable	Purpose
<code>general_model</code>	Pretrained YOLOv8m model — detects 80 everyday COCO objects.
<code>helmet_model</code>	Custom-trained model — detects helmet / no-helmet.
<code>line_x</code>	X-position of the vertical counting line (pixel 320).
<code>crossed_ids</code>	Set of tracked person IDs already counted, to prevent double counting.
<code>total_count</code>	Running total of people who crossed the line.
<code>object_counts</code>	Per-frame tally of how many of each object class are visible.
<code>fps</code>	Live frames-per-second, recalculated every processed frame.
<code>out</code>	VideoWriter object saving the annotated output to <code>output.avi</code> .

This explanation is based directly on the `main.py` code you provided — every section above matches the actual logic in your script.